

Day 46 : React Hook Form

React Hook Form & Zod - Quick Notes

Why React Hook Form?

Traditional Problem:

```
// ❌ Too many states, re-renders on every keystroke
const [name, setName] = useState('')
const [email, setEmail] = useState('')
const [password, setPassword] = useState('')
```

RHF Solution:

```
// ✅ No states, no re-renders, clean code
const { register, handleSubmit } = useForm()
```

Basic Setup

```
npm install react-hook-form
```

```
import { useForm } from 'react-hook-form'

function App() {
  const { register, handleSubmit } = useForm()

  const onSubmit = (data) => {
    console.log(data) // { name: "...", email: "..." }
  }
}
```

```

return (
  <form onSubmit={handleSubmit(onSubmit)}>
    <input {...register('name')} />
    <input {...register('email')} />
    <button type="submit">Submit</button>
  </form>
)
}

```

Built-in Validation

```

const { register, handleSubmit, formState: { errors } } = useForm()

<input
  {...register('name', {
    required: 'Name required',
    minLength: { value: 3, message: 'Min 3 chars' },
    maxLength: { value: 20, message: 'Max 20 chars' }
  })}
/>
{errors.name && <span>{errors.name.message}</span>}

<input
  {...register('email', {
    required: 'Email required',
    pattern: { value: /\S+@\S+\.\S+/, message: 'Invalid email' }
  })}
/>

<input
  type="number"
  {...register('age', {
    min: { value: 18, message: 'Min 18' },

```

```
    max: { value: 100, message: 'Max 100' },  
    valueAsNumber: true  
  })  
  />
```

Zod Validation (Better)

```
npm install zod @hookform/resolvers
```

```
import { zodResolver } from '@hookform/resolvers/zod'  
import { z } from 'zod'  
  
const schema = z.object({  
  name: z.string().min(3, 'Min 3').max(20, 'Max 20'),  
  email: z.string().email('Invalid email'),  
  password: z.string().min(6, 'Min 6 chars')  
    .regex(/[A-Z]/, 'Need uppercase')  
    .regex(/[0-9]/, 'Need number'),  
  age: z.number().min(18, 'Min 18').max(100, 'Max 100')  
})  
  
const { register, handleSubmit, formState: { errors } } = useForm({  
  resolver: zodResolver(schema)  
})  
  
// Use normally  
<input {...register('name')} />  
{errors.name && <span>{errors.name.message}</span>}
```

Zod Types:

- `z.string()` - Text
- `z.number()` - Numbers

- `z.string().email()` - Email
 - `z.string().optional()` - Optional field
-

Password Confirmation

```
const schema = z.object({  
  password: z.string().min(6),  
  confirmPassword: z.string()  
}).refine((data) => data.password === data.confirmPassword, {  
  message: "Passwords don't match",  
  path: ["confirmPassword"]  
})
```

Useful Features

Default Values

```
const { register } = useForm({  
  defaultValues: {  
    name: 'Rohit',  
    email: 'rohit@email.com'  
  }  
})
```

watch() - Conditional Fields

```
const { register, watch } = useForm()  
const role = watch('role')  
  
<select {...register('role')}>  
  <option value="student">Student</option>  
  <option value="professional">Professional</option>  
</select>
```

```
{role === 'student' && <input {...register('college')} />}  
{role === 'professional' && <input {...register('company')} />}
```

reset() - Clear Form

```
const { reset } = useForm()  
  
const onSubmit = (data) => {  
  console.log(data)  
  reset() // Clear form  
}
```

setValue() - Set Values

```
const { setValue } = useForm()  
  
setValue('name', 'John')  
setValue('email', 'john@email.com')
```

isSubmitting - Loading State

```
const { formState: { isSubmitting } } = useForm()  
  
<button disabled={isSubmitting}>  
  {isSubmitting ? 'Loading...' : 'Submit'}  
</button>
```

Validation Modes

```
const { register } = useForm({  
  mode: 'onBlur' // Best for UX  
})
```

```
})
```

- `onSubmit` - Default, validates on submit
- `onChange` - Validates on every keystroke
- `onBlur` - Validates when leaving field ★ Recommended

Complete Example

```
import { useForm } from 'react-hook-form'
import { zodResolver } from '@hookform/resolvers/zod'
import { z } from 'zod'

const schema = z.object({
  name: z.string().min(3).max(20),
  email: z.string().email(),
  password: z.string().min(6),
  confirmPassword: z.string()
}).refine(data => data.password === data.confirmPassword, {
  message: "Passwords don't match",
  path: ["confirmPassword"]
})

function App() {
  const {
    register,
    handleSubmit,
    reset,
    formState: { errors, isSubmitting }
  } = useForm({
    resolver: zodResolver(schema),
    mode: 'onBlur'
  })

  const onSubmit = async (data) => {
```

```

    await fetch('/api/submit', { method: 'POST', body: JSON.stringify(data) })
    reset()
  }

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <div>
        <input {...register('name')} placeholder="Name" />
        {errors.name && <span>{errors.name.message}</span>}
      </div>

      <div>
        <input {...register('email')} placeholder="Email" />
        {errors.email && <span>{errors.email.message}</span>}
      </div>

      <div>
        <input type="password" {...register('password')} placeholder="Password" />
        {errors.password && <span>{errors.password.message}</span>}
      </div>

      <div>
        <input type="password" {...register('confirmPassword')} placeholder="Confirm Password" />
        {errors.confirmPassword && <span>{errors.confirmPassword.message}</span>}
      </div>

      <button disabled={isSubmitting}>
        {isSubmitting ? 'Submitting...' : 'Submit'}
      </button>
    </form>
  )
}

```

Key Benefits

- ✓ No `useState` for each field
 - ✓ No re-renders while typing
 - ✓ Less boilerplate code
 - ✓ Built-in validation
 - ✓ Easy error handling
 - ✓ Zod makes validation cleaner
-

Quick Reference

```
// Setup
const { register, handleSubmit, watch, reset, setValue, formState } = useForm
({
  resolver: zodResolver(schema),
  mode: 'onBlur',
  defaultValues: {}
})

// Access
formState.errors    // Validation errors
formState.isSubmitting // Submission state

// Methods
register('name')    // Connect input
handleSubmit(fn)    // Handle submit
watch('field')      // Read field value
reset()             // Clear form
setValue('field', value) // Set value
```